PocketSmart AI

Smart Budget and Recommendation Assistant

Home Interior | Party | Jewelry

A Generative AI based web application for budget-wise planning

Student Name	
Register Number	
College / Institution	
Program	
Guide / Mentor	
Academic Year	

Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Scope of the Project
6. Existing System and Proposed System
7. System Modules
8. System Architecture
9. Technology Stack
10. Database Design
11. API Endpoints
12. Workflow
13. Testing
14. Advantages and Limitations
15. Future Enhancements
16. Conclusion

1. Abstract

Planning a purchase or an event within a fixed budget is a common problem. People spend hours searching different shopping and service websites and calculating costs by hand, and they still overspend. **PocketSmart AI** is a web application that takes the user's budget and requirements and uses Generative AI to divide the money across categories and suggest what to buy or book.

The system has three planners: **Home Interior**, **Party** and **Jewelry**. Each planner produces a category-wise budget split and gives direct search links to popular platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato and OYO. The Jewelry planner can also read an uploaded outfit photo and suggest matching jewelry. Users can register, log in and view their earlier recommendations. The application is built with FastAPI, Jinja2 and the Google Gemini API, and it includes a fallback engine so that it keeps working even when the AI service is unavailable.

2. Introduction

Online shopping and event booking have made choices unlimited, but this also makes decisions harder. A family furnishing a new house, a student organising a birthday party or a person buying jewelry for a wedding all face the same question: *how should I divide my money so that I get the best result?*

Generative AI models can understand a request written in plain language and produce structured suggestions. PocketSmart AI uses this ability to act as a personal budget assistant. The user only enters a budget and a few details, and the system returns a clear plan.

3. Problem Statement

Users do not have a single, easy tool that converts a budget into a practical spending plan across different needs. Existing shopping sites show products but do not plan the whole budget, and generic expense trackers record money already spent rather than guiding future spending. The problems are:

- Time is wasted comparing products and prices across many websites.
- Budget is divided without a proper basis, so some categories get too much and others too little.
- There is no help in matching items with each other, for example jewelry with an outfit.
- Past plans are not saved, so the user has to start again every time.

4. Objectives

- To build an AI assistant that splits a given budget across categories for home interior, party and jewelry planning.
- To give relevant platform links so the user can quickly search and buy or book.

- To support image-based jewelry suggestions by reading the colour and style of an uploaded outfit.
- To provide secure registration, login and a history of past recommendations.
- To keep the application usable even without an AI connection through a rule-based fallback.

5. Scope of the Project

In scope: budget planning and recommendations for the three modules, user accounts, history, and platform search links.

Out of scope: live price or stock lookup from shopping sites, online payment, and order placement. The system suggests and guides; the actual purchase is done by the user on the chosen platform.

6. Existing System and Proposed System

Point	Existing approach	PocketSmart AI
Budget planning	Manual calculation in notebook or spreadsheet	AI divides the budget automatically
Product search	Open many websites one by one	One plan with direct platform links
Matching items	User's own guess	AI reads outfit image and suggests matching jewelry
History	Not available	Every recommendation is saved per user
Availability	Depends on internet tools	Fallback engine works without AI key

7. System Modules

7.1 User Authentication

New users register with name, email and password. Passwords are stored in hashed form. On login the server issues a token, and protected pages such as history are opened only for logged-in users.

7.2 Home Interior Planner

The user enters the total budget and the rooms or items to be furnished (lights, fans, furniture, dining items, decor). The AI divides the amount across the items and suggests suitable options, with search links to Amazon, Flipkart, IKEA and similar stores.

7.3 Party Planner

The user enters budget, number of guests and type of party. The budget is divided among catering, venue or stay, decoration, entertainment and a small contingency. Links are provided for food delivery, stay booking and decoration items.

7.4 Jewelry Planner

The user gives a budget and an occasion, and can optionally upload a photo of the outfit. The AI checks the colours and style and recommends necklaces, earrings, bangles or rings within the budget.

7.5 Recommendation History

Every result is stored against the user account so it can be opened again later without re-entering details.

7.6 AI Service and Fallback Engine

The AI service builds a prompt from the user's input, calls the Gemini API and formats the reply for the page. If the API key is missing or the call fails, the fallback engine gives a rule-based split so the user still gets a plan.

8. System Architecture

The application follows a simple three-layer design.

Layer	Components	Role
Presentation	HTML, CSS, JavaScript pages rendered with Jinja2	Forms for input, pages to show results and history
Application	FastAPI routes, authentication, planner logic	Validates input, calls AI, builds response
AI	Google Gemini API + fallback engine	Budget split and recommendations, image understanding
Data	SQLite database	Stores users and saved recommendations

Flow: User -> Web page -> FastAPI route -> Planner logic -> Gemini API (or fallback) -> Result page -> Saved in history.

9. Technology Stack

Area	Technology	Why it is used
Backend	Python, FastAPI, Uvicorn	Fast, simple REST framework with automatic validation
AI	Google Gemini API	Understands text and images, gives structured output
Frontend	Jinja2, HTML5, CSS3, JavaScript	Server-side pages that are easy to build and test
Database	SQLite	Lightweight, no separate server needed
Security	JWT tokens, hashed passwords	Safe login and session handling
Images	Pillow	Reads and prepares the uploaded outfit image

10. Database Design

Two main tables are used.

Table	Fields	Description
users	id, name, email, password_hash, created_at	One row for each registered user
recommendations	id, user_id, module, budget, input_details, result, created_at	One row for each saved plan; user_id links to users

11. API Endpoints

Sample route design. Names can be adjusted to match your final code.

Method	Route	Purpose
POST	/register	Create a new account
POST	/login	Verify user and issue token
GET	/logout	End the session
POST	/home-interior	Generate home interior budget plan
POST	/party	Generate party budget plan
POST	/jewelry	Generate jewelry suggestions (text and optional image)
GET	/history	List saved recommendations of the user

12. Workflow

- User registers and logs in.
- User chooses a planner: Home Interior, Party or Jewelry.
- User enters budget and details (and an outfit image for jewelry, if wanted).
- Server validates the input and prepares a prompt for the AI.
- Gemini returns the budget split and suggestions; if it fails, the fallback engine answers.
- The result page shows the plan with platform search links.
- The plan is saved in history for later viewing.

13. Testing

Testing covers login, each planner, image upload, history, and behaviour without an AI key. The accompanying Excel workbook lists 14 test cases with input and expected result; record the actual outcome (Pass or Fail) after running them on your own system.

Area	Example test	Expected result
Authentication	Login with wrong password	Error shown, no session created
Home Interior	Very low budget	Message asking for a higher budget
Party	Guest count left blank	Validation error
Jewelry	Non-image file uploaded	Only image files allowed message
Security	Open history without login	Redirect to login page
AI	No API key configured	Fallback plan still displayed

14. Advantages and Limitations

Advantages

- Saves time by giving one complete plan instead of many searches.
- Simple to use; only budget and a few details are needed.
- Works for three different everyday needs in one application.
- Continues to work with the fallback engine when AI is not reachable.

Limitations

- Does not fetch live prices or stock; links only open search pages.
- AI suggestions are guidance and may not match exact market rates.
- Image matching depends on the quality of the uploaded photo.

15. Future Enhancements

- Live price comparison through official partner APIs.
- Support for more planners such as travel, wedding and gadgets.
- Mobile application and Tamil language support.
- Expense tracking to compare planned and actual spending.
- Sharing a plan with family members.

16. Conclusion

PocketSmart AI shows how Generative AI can turn a simple budget into a useful spending plan. By combining a clean web interface, secure user accounts, image-aware recommendations and a reliable fallback, the project offers a practical tool for everyday planning and can be extended into a full personal finance assistant.